Atividade — Code Smells e Refatoração

Integrante: Lucas Muller Scuzziato
LLM: ChatGPT

---

1. Long Method

Benefício: funções menores são mais fáceis de ler, testar e alterar.

Como refatorar: extrair blocos com responsabilidade clara para funções nomeadas (Extract Method); a função original só orquestra.

Com smell:

```
def processar_pedido(itens, cupom):
    total = 0
    for item in itens:
        if item["qtd"] <= 0 or item["preco"] < 0:
            raise ValueError("item inválido")
        total += item["qtd"] * item["preco"]

    if cupom == "DESC10":
        total = total * 0.9
    elif cupom == "DESC20":
        total = total * 0.8

    linhas = []
    for item in itens:
        linhas.append(f"{item['nome']}: {item['qtd']} x {item['preco']:.2f}")
    linhas.append(f"Total: R$ {total:.2f}")
    return "\n".join(linhas)
```

Refatorado:

```
def validar_itens(itens):
    for item in itens:
        if item["qtd"] <= 0 or item["preco"] < 0:
            raise ValueError("item inválido")


def calcular_subtotal(itens):
    return sum(item["qtd"] * item["preco"] for item in itens)


def aplicar_desconto(total, cupom):
    descontos = {"DESC10": 0.9, "DESC20": 0.8}
    return total * descontos.get(cupom, 1.0)


def formatar_recibo(itens, total):
    linhas = [
        f"{item['nome']}: {item['qtd']} x {item['preco']:.2f}"
        for item in itens
    ]
    linhas.append(f"Total: R$ {total:.2f}")
    return "\n".join(linhas)


def processar_pedido(itens, cupom):
    validar_itens(itens)
    total = aplicar_desconto(calcular_subtotal(itens), cupom)
```

```
    return formatar_recibo(itens, total)
```

---

2. Duplicate Code

Benefício: correção e mudança de regra em um só lugar; menos inconsistência.

Como refatorar: extrair a lógica comum para uma função compartilhada e substituir as cópias por chamadas.

Com smell:

```
def media_turma_a(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)


def media_turma_b(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)


def media_turma_c(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)
```

Refatorado:

```
def media(notas):
    if not notas:
        return 0.0
    return sum(notas) / len(notas)
```

---

3. Magic Numbers

Benefício: o valor ganha significado; mudança de regra fica centralizada.

Como refatorar: nomear literais importantes como constantes e reutilizá-las.

Com smell:

```
def calcular_frete(peso_kg, distancia_km):
    if peso_kg > 5:
        return 12 + distancia_km * 0.15
    return 2.5 + distancia_km * 0.15
```

Refatorado:

```
PESO_LIMITE_KG = 5
TAXA_POR_KM = 0.15
TAXA_BASE_LEVE = 2.5
TAXA_BASE_PESADO = 12.0


def calcular_frete(peso_kg, distancia_km):
    taxa_base = TAXA_BASE_PESADO if peso_kg > PESO_LIMITE_KG else TAXA_BASE_LEVE
    return taxa_base + distancia_km * TAXA_POR_KM
```